The AI Stack

From infrastructure to application

A map of how AI systems are built — and where the money, the work, and the value actually are.

Agenda

1. **Why a stack?** — a model is one piece
2. **The 5 layers** — infra → models → data → orchestration → application
3. **The business view** — revenue pyramid · 3 types of innovation
4. **Case studies** — chatbot · RAG · the vendor-vs-operator trap
5. **GenAI for DevOps** — incident RCA · observability · IaC · CI/CD risk
6. **Takeaways**

Why a "stack"?

A model is **one piece**, not the whole system.

To solve a real problem you also need: **compute** to run it, **data** to ground it, **orchestration** to coordinate it, and an **application** for the user.

Every choice across the stack drives **quality · speed · cost · safety**.

The 5 layers

```
5 · APPLICATION     interfaces & integrations — the user
4 · ORCHESTRATION   plan → execute → review — the "brain" (agentic)
3 · DATA            sources, pipelines, vector store, RAG
2 · MODELS          open/proprietary · size · specialization
1 · INFRASTRUCTURE  GPUs — on-premise, cloud, local
```

We'll go bottom → top, then look at the **business** picture.

1 · Infrastructure

LLMs need **AI-specific hardware (GPUs)**.

Three ways to deploy:

- **On-premise** — own it; full control, high upfront cost
- **Cloud** — rent it; scale up/down on demand
- **Local** — laptop; only the smaller models

> The hardware you can access decides what models you can run at all.

2 · Models

Pick along three dimensions:

- **Open vs proprietary** — control, cost, where it runs
- **Size** — large (LLM) vs small (SLM)
- **Specialization** — reasoning, tool-calling, code, language

Use vs build: consume a model, **fine-tune** it, or (rarely) **train** it.
Lifecycle glue = **MLOps**.

> Need fresh knowledge, not new behavior? Prefer RAG over fine-tuning.

3 · Data

Base models have a **knowledge cutoff** and don't know your private data.

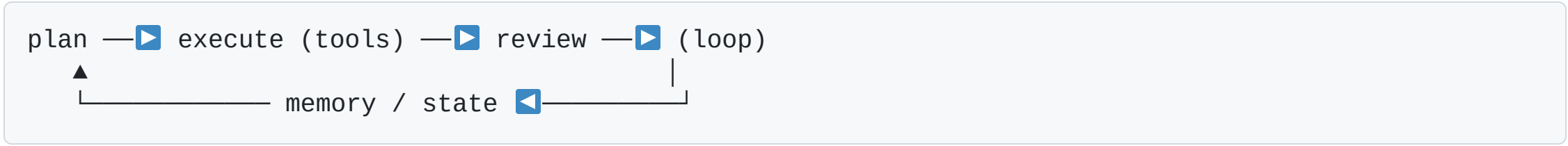
```
sources → pipelines (clean/chunk) → embed → vector store → RAG → model
```

RAG retrieves relevant context and augments the prompt — fresh & private knowledge without retraining.

4 · Orchestration — the agentic layer

One prompt in / one answer out = a **chat box**.

Add the loop and it becomes an **agent**:



Two kinds: **static** (reliable workflows) + **agentic** (autonomous decisions).

Fastest-moving layer — agents, MCP.

5 · Application

Where AI meets the user. Two questions:

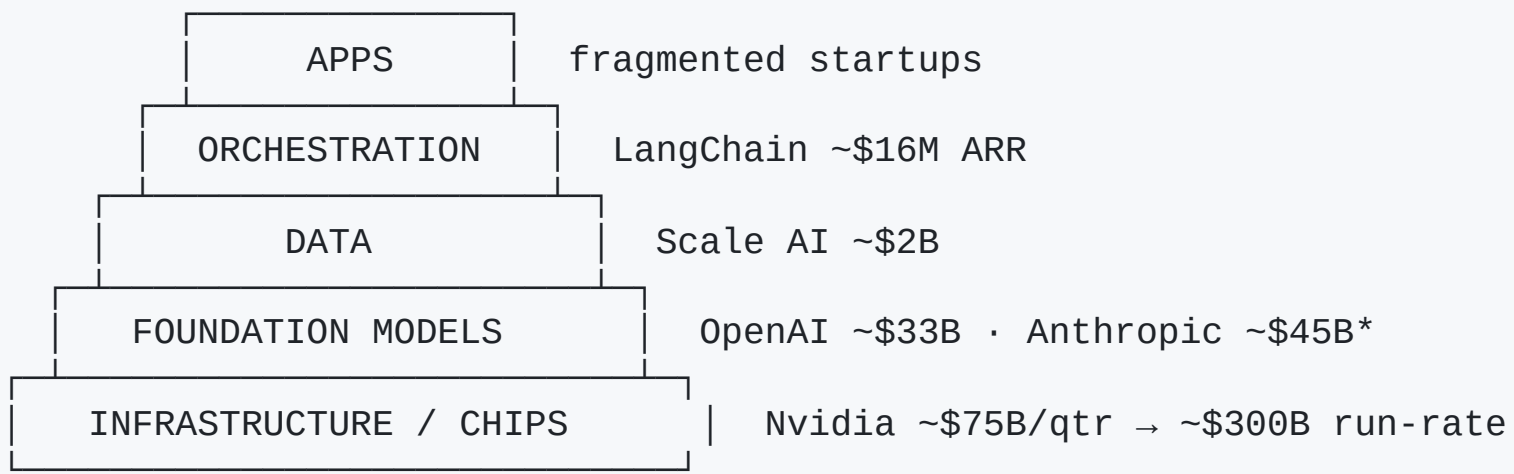
- **Interfaces** — text / image / audio · revisions · citations · *conversational UI replacing forms*
- **Integrations** — inbound (tools feed AI) & outbound (AI output → tools)

> "What the AI *does*" is orchestration. "What the user sees / where the result lands" is the application layer.

The business view

Where is the money — and where is the value?

Revenue pyramid (mid-2026)



Revenue concentrates at the **base** (Nvidia alone > all model companies).

Hype ≠ revenue — orchestration gets the most attention, the least money (LangChain ≈ 1/18,000 of Nvidia).

Three types of innovation

Clayton Christensen — a lens for any AI product

Type	Idea	Jobs	AI example
Market-creating	expensive/hard → cheap for all	creates	foundation models for everyone
Sustaining	make a good product better	neutral	an AI feature in your app
Efficiency	same work, less cost	reduces	coding assistants, automation

The type isn't the *layer* — it's how the tech is **used**.

Case · sales / support chatbot

Klarna × OpenAI: 2.3M chats in month 1 = **700 agents'** work,

67% automated, 11 min → **<2 min**, **~$40M** profit (2024).

Caveat: cut too deep, rehired in 2025 — AI-first, not AI-only.

```
ILLUSTRATIVE — 100,000 contacts/mo · human $5 · AI $1 · 67% deflection
  AI handles  67,000 × $1 = $67k/mo
  vs humans   67,000 × $5 = $335k/mo
  net saving  ≈ $268k/mo  ≈ $3.2M/yr
```

Players: Sierra · Decagon · Ada · Intercom Fin

Case · RAG over internal docs

Morgan Stanley × OpenAI

- Indexed **350,000 documents (40M words)** with RAG
- Before: **30+ min** manual search · advisors reached **~20%** of knowledge
- After: instant · **98%** of teams use it · access **20%** → **80%**

Frees advisor time for client work; answers grounded in verified sources.

Similar: Glean · Harvey · Hebbia

The trap: vendor value vs operator value

The pyramid measures who earns money **selling** AI → base wins.

Case studies measure value from **applying** AI → lands on the *operator's* books.

> Klarna's $40M isn't an "AI app company" revenue line — it's on **Klarna's** P&L. The app layer looks thin for vendors, yet is where operators win.

GenAI for DevOps

An AI agent for ops = the whole stack

```
5 · APPLICATION    Slack on-call copilot · NL query · Datadog/PagerDuty
4 · ORCHESTRATION  agent: query logs/metrics/traces → traverse deps → RCA
3 · DATA           vector DB (FAISS/Weaviate): telemetry + incidents + runbooks
2 · MODELS         LLM, prompt-engineered/fine-tuned for logs & remediation
1 · INFRASTRUCTURE GPUs to run it
```

This is *why* we learned the stack: a production SRE assistant exercises **all five layers** at once.

Incident triage, RCA & remediation

An AI agent watches alerts, reads logs / metrics / traces, and proposes a root cause and a fix.

Tools: Cleric · Resolve.ai · Traversal · Rootly · incident.io · Datadog Bits AI

Proven in production:

Where	Result
Traversal @ American Express	82% RCA accuracy · −32% MTTR
Microsoft RCACopilot	~0.77 RCA accuracy · 4 yrs, 30+ teams

> Vendor "38–90% MTTR" claims are self-reported. Remediation stays **human-in-the-loop**.

Observability: ask telemetry in English

NL querying + anomaly detection on top of your existing tools (Datadog, Prometheus, Grafana, OpenTelemetry):

- **Datadog Bits Assistant** — query dashboards/logs/traces in plain language
- **Grafana Assistant** — NL telemetry questions + ML correlation
- **Datadog Toto** — timeseries foundation model powering anomaly forecasting

Grounding = **RAG over telemetry + incident history + runbooks** in a vector DB (FAISS / Weaviate), so the LLM reasons over *your* system, not generic text.

IaC with AI — the real risk

The problem isn't *bad* code. It's **plausible** code — code nobody reads line-by-line anymore.

A 200-line Terraform module, AI-written in **30 s**. The dev skims it, sees no syntax errors, runs `apply`. What goes unchecked:

- Security group with ingress `0.0.0.0/0`?
- S3 bucket missing a public-access block?
- RDS without encryption-at-rest? · IAM policy with `Resource: "*"`?
- Secrets hardcoded in `tfvars`?

> **Anti-pattern:** AI generates Terraform → `apply` → 💥
> (no review, no scan, no policy gate)

What AI tends to get wrong

Common misconfigurations in AI-generated Terraform (approx.):

#	Misconfiguration	Seen in	Why
1	Security group `0.0.0.0/0` on 22/3389	~25%	common demo examples
2	S3 missing public-access block	~20%	not blocked by default
3	RDS not encrypted	~18%	encryption is opt-in
4	IAM `Resource: "*"`	~15%	convenient in demos
5	Secrets in `tfvars` (no SOPS)	~12%	pattern is in training data
6	Missing tags	~30%	tag policy is org-specific
7	Hardcoded creds in provider block	~5%	rarer, but critical

The 3-layer defense

```
1 · AI GENERATE    Claude Code + Terraform MCP
                   constraint-first, security-first prompts
        ↓
2 · HUMAN REVIEW   read the plan, understand every resource
                   ask the agent what you don't get — never approve blindly
        ↓
3 · POLICY GATE    tflint → checkov → terraform plan → conftest
                   → AI explains the diff → human approves → apply
                   RULE: checkov not clean (any HIGH) → no apply
```

L1 is fast but **plausible-but-wrong** · L2 catches *intent*, but humans tire ·

L3 is automated & consistent — it catches what humans miss.

CI/CD risk & guardrails (the frontier)

Emerging, less mature than incident response:

- **Blast-radius analysis** — LLM predicts what a change can break
- **Risk scoring** of config / schema changes *before* rollout
- **Auto validation & rollback** informed by historical outcomes

> Honest take: fewer proven products here than in RCA/observability — strong territory for a platform team to build real differentiation.

Takeaways

1. AI is a **stack**, not a model — infra → models → data → orchestration → app
2. Money sits at the **base**; **value** from *applying* AI sits with the **operator**
3. Name the innovation type: **market-creating / sustaining / efficiency**
4. Always do the math with **both sides** — savings *and* the AI's own cost
5. In DevOps, **AI drafts — your pipeline verifies**

Thank you

Questions?

Speaker notes & full numbers: `README.md` *and* `docs/devops-cases.md`

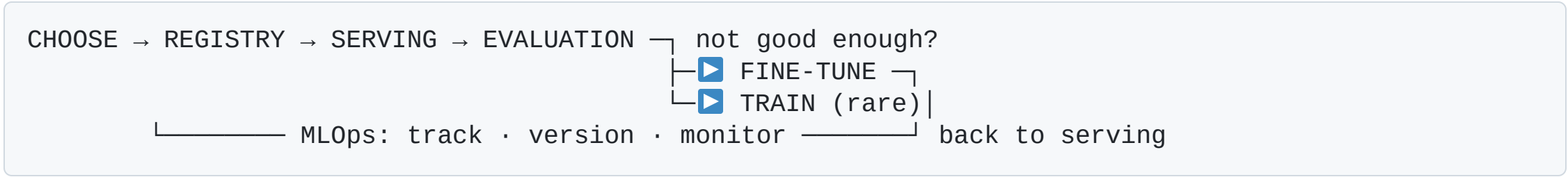
Models — deep dive

Layer 2, hands-on (lab day)

Choose → serve → evaluate → fine-tune → operate, on one RTX 5090.

The model lifecycle

A model isn't a file you download once — it's a loop:



Two rules we keep returning to:

- Need new **knowledge**? → **RAG** (layer 3), not the model.
- Need new **behavior**? → **fine-tune**. Train from scratch almost never.

Choosing — three dimensions

- **Open vs proprietary** — runs on your hardware vs vendor API; control & privacy vs frontier ceiling
- **Size (LLM vs SLM)** — bigger = smarter but heavier; smaller = cheaper + often specialized
- **Specialization** — reasoning · tool-calling · code · language · multimodal

> "Open-weights" ≠ "open-source" — you get weights, not always data or a free licence. **Read the licence.**

The one number: VRAM

```
VRAM(weights) ≈ params × bytes   (fp16=2 · fp8=1 · int4≈0.5)
```

On our **32 GB** RTX 5090:

size	fp16	int4 (Q4)	fp16 fits?	Q4 fits?
7–8B	~15 GB	~5 GB	✅	✅
14B	~28 GB	~8 GB	⚠️ tight	✅
32B	~64 GB	~18 GB	❌	✅

This table *is* why quantization exists — and why a 32 GB card runs a 32B model.

Serving — two engines, measure the difference

	Ollama / llama.cpp	vLLM
Best for	one user, demos	many users, throughput
Trick	GGUF + Q4 built in	PagedAttention + batching
Feel	snappy single answer	wins at concurrency

> **Latency** = one fast answer · **throughput** = total tokens/s for everyone.

Lab 1: one model (Qwen2.5-3B) at **fp16 vs Q4** on Ollama — Q4 ≈ ½ VRAM, ~1.5× speed. (vLLM optional, for the batching contrast.)

Fine-tuning — teach behavior, cheaply

Full fine-tune of 7B = weights + grads + optimizer ≈ tens of GB → not on one card.

- **LoRA** — freeze the base, train tiny adapters (<1% of params)
- **QLoRA** — load the base in **4-bit**, train the adapter on top

method	base	VRAM (7B)
LoRA	bf16	~18–24 GB
QLoRA	4-bit	~8–12 GB

> You're not moving 7B weights — you learn a few million adapter numbers.

Lab 2: turn Qwen2.5-3B into a strict-JSON order parser; track in **MLflow**.

Evaluation — numbers, not vibes

kind	what
Benchmark	MMLU / GSM8K / ARC — general ability
Task eval	*your* held-out set + a metric you chose
LLM-judge / human	preference, helpfulness

> Pitfalls: **contamination** (memorized benchmarks), Q4 quality (measure it), and "one number lies" — report quality **+** speed **+** cost together.

Lab 3: base vs fine-tuned on JSON-valid % / exact-match / field accuracy.

Training & MLOps — the bookends

- **Training from scratch:** trillions of tokens + a cluster (that's why Layer 1 exists). Millions of dollars, months. We *explain*, we don't run it.
- **MLOps** is the glue: experiment tracking, model/dataset versioning, eval gates in CI, drift monitoring.

> Decision tree, almost always: **use** → **fine-tune** → **(only then) train.**

Models — takeaways

1. It's a **lifecycle**, not a download: choose → serve → eval → adapt → operate
2. **VRAM** governs what runs; **quantization** is the lever
3. **Fine-tune for behavior, RAG for knowledge** — train almost never
4. **Measure** everything — quality *and* speed *and* cost
5. **MLOps** makes it reproducible